МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«УФИМСКИЙ ГОСУДАРСТВЕННЫЙ НЕФТЯНОЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра «Цифровых технологий и моделирования»

Направление подготовки
09.04.01 – «Информатика и вычислительная техника»
профиль «Искусственный интеллект и цифровые двойники»

СИСТЕМА ПРЕДИКТИВНОЙ АНАЛИТИКИ УСТАЛОСТИ ОПЕРАТОРА В VR С ИСПОЛЬЗОВАНИЕМ ОКУЛОГРАФИИ И ЭЭГ

Курсовая работа

Работа защищена
«___» _____ 20__ г.
Оценка _____
(подпись)

Выполнил студент
2 курса группы МПИ09-25
Казаккулов Урал Фанилевич

Научный руководитель
канд. наук, доц. Чиганова Н.В.
_____
(подпись)

Уфа – 2026 г.

СОДЕРЖАНИЕ

Введение 3
1 Теоретические основы оценки состояния оператора 4
1.1 Утомление и когнитивная нагрузка 4
1.2 Поведенческие, окулографические и нейрофизиологические признаки 4
1.3 Принцип работы разрабатываемой системы 5
2 Проектирование и реализация системы предиктивной аналитики 7
2.1 Архитектура системы и обмен данными между Unity и Python 7
2.2 Реализация тестовой виртуальной среды и регистрация поведения оператора 10
2.3 Обработка метрик Emotiv, расчет интегрального индекса и режимы работы системы 13
Заключение 18
Список использованных источников 19

ВВЕДЕНИЕ

Виртуальная реальность используется для обучения и моделирования операторских задач, в которых от пользователя требуется устойчивое внимание и быстрая реакция. При длительной работе могут увеличиваться время ответа, количество ошибок и число отвлечений. Поэтому оценка состояния оператора является актуальной задачей.

Нейроинтерфейсы позволяют учитывать состояние пользователя при взаимодействии с виртуальной средой: применяться показатели выполнения задания, параметры взгляда и различные метрики Emotiv.

Цель курсовой работы — разработать прототип системы предиктивной аналитики состояния оператора в виртуальной среде на основе поведенческих, окулографических и доступных метрик Emotiv.

Для достижения цели необходимо решить следующие задачи:

1. Рассмотреть признаки когнитивной нагрузки и утомления оператора.
2. Определить состав данных во время выполнения задания.
3. Разработать архитектуру взаимодействия Unity и Python-сервиса.
4. Реализовать виртуальную сцену с тестовым заданием, таймером и регистрацией ответов.
5. Добавить моделирование окулографических событий и обработку метрик Emotiv.
6. Разработать расчет интегрального индекса признаков утомления и его классификацию по трем категориям.

Объект исследования — процесс выполнения оператором задания в виртуальной среде.

Предмет исследования — методы объединения поведенческих, окулографических и нейрофизиологических признаков для оценки состояния оператора.

1 Теоретические основы оценки состояния оператора

1.1 Утомление и когнитивная нагрузка

При выполнении каких-либо операторских задач человек воспринимает какую-либо информацию, выбирает действие и проверяет его результаты. Чем больше операций ему нужно выполнить за ограниченное время, тем больше нагрузка на внимание и рабочую память [1, 2]. В течение рабочей сессии это может проявляться увеличением времени реакции, ошибками и пропусками сигналов [1]. Один такой признак не позволяет сделать вывод об утомлении, поэтому в прототипе должна учитываться их совокупность [2].

В работе утомление рассматривается как изменение качества выполнения задания во времени. Интегральный индекс признаков утомления – числовая оценка учебного прототипа, а не медицинское заключение. Это обуславливается тем, что одна из задач курсовой работы – проверить конвейер регистрации, передачи и агрегации данных [9].

1.2 Поведенческие, окулографические и нейрофизиологические признаки

Одним из способов оценки поведенческих признаков является – тест, из него мы можем получить: время реакции, правильность ответа и пропуски. Они понятны для проверки, но зависят от сложности задания и опыта пользователя [3, 4]. В прототипе поведенческий канал фиксирует время реакции на тревогу, долю правильных решений, число ошибок и пропусков, а также количество поздних правильных решений. Именно рост времени реакции и появление пропусков при сохранении темпа считаются ранними признаками утомления, тогда как отдельная ошибка ещё не говорит об ухудшении состояния [4].

Окулографические признаки описывают фиксации, разброс взгляда и отвлечения [3, 4]. В текущей версии отдельный eye-tracker будет отсутствовать, так как нет соответсвенного оборудования под рукой, поэтому взгляд моделируется лучом из камеры Unity. Модель подходит для проверки обработки событий и не заменяет измерение движений глаз, и имитирует работу eye-tracker [3].

Нейрофизиологические признаки поступают от Emotiv только после проверки фактического сообщения Cortex: используются Performance Metrics из потока met [5, 9].

Перечисленные признаки имеют разную природу и разные ограничения при практическом использовании. Для наглядного сопоставления они сведены в таблицу 1.

Таблица 1 — Группы признаков и их ограничения

Группа данных	**Примеры**	**Ограничение**
Поведение	Время реакции, ошибки, пропуски	Зависит от задания
Взгляд	Фиксация, разброс, отвлечение	В проекте моделируется
Emotiv Cortex	Только подтверждённые метрики	Зависит от конфигурации

Таким образом, три группы признаков дополняют друг друга: поведение отражает результат действия, взгляд — распределение внимания, а Emotiv – косвенную оценку вовлечённости [5, 6].

1.3 Принцип работы разрабатываемой системы

Пользователь выполняет короткий интерактивный тест в Unity. После каждого действия приложение сразу показывает обратную связь, запускает

следующий раунд и отправляет событие в Python. Сервис объединяет ответы, события виртуального взгляда и доступные метрики источника, после чего возвращает результат в Unity. Пользовательский интерфейс остаётся в Unity, а сбор и расчёт выполняет отдельный аналитический сервис [7].

Интерактивность теста состоит в повторяющемся цикле: оператор выбирает действие, получает немедленную обратную связь и переходит к следующему событию. В это время система фиксирует реакцию и виртуальный взгляд [3, 7]. После завершения сессии Unity показывает индекс, категорию и основные признаки.

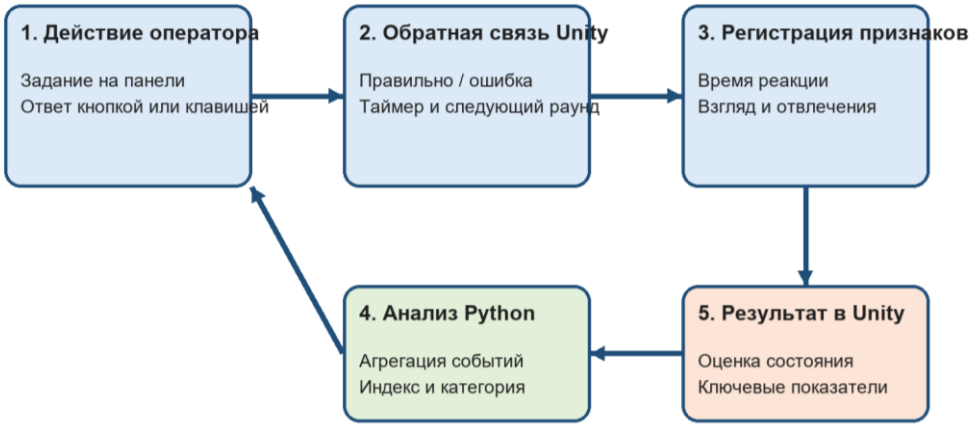


Рисунок 1 – Интерактивный цикл тестовой сессии

Для первой версии выбран эвристический расчёт: его результат можно воспроизвести и связать с входными признаками. Эвристический расчёт учитывает фазу события и доступный лимит времени, чтобы короткие лимиты кризисной фазы не интерпретировались как ухудшение состояния сами по себе [6]. Итоговый индекс сопровождается категорией low / medium / high и техническим разбором признаков; он является оценкой учебного прототипа, а не медицинским заключением.

2 Проектирование и реализация системы предиктивной аналитики

2.1 Архитектура системы и обмен данными между Unity и Python

Разрабатываемая система построена по клиент-серверной схеме. Пользовательская часть реализована в Unity, а сбор и обработка аналитических данных выполняются отдельным Python-сервисом. Такое разделение позволяет не связывать логику виртуальной среды с расчётом итогового показателя состояния оператора [7].

Unity отвечает за весь пользовательский сценарий. В приложении находятся стартовое окно, операторская сцена, интерактивные задания, таймер, элементы управления и экран результатов. Python не формирует пользовательский интерфейс. Его задача состоит в приёме событий, подключении к Cortex API, накоплении данных и расчёте итоговых признаков.

В системе используются три основных источника данных:

- поведенческие события, формируемые Unity;
- моделируемые события взгляда;
- поток Performance Metrics Emotiv от виртуального устройства EPOC X.

Общая схема взаимодействия имеет следующий вид:

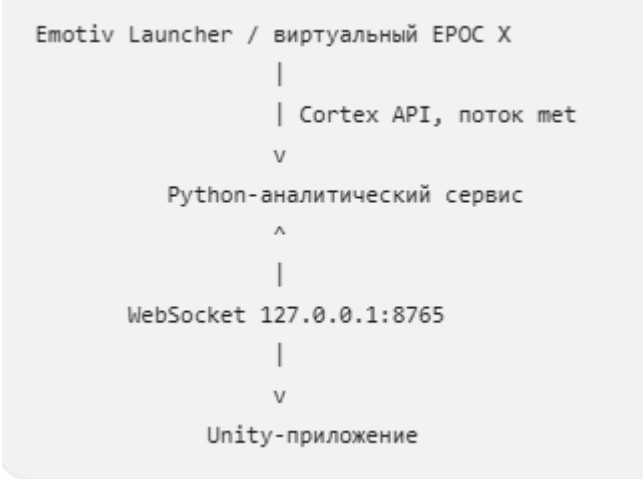


Рисунок 2 – Схема пути источников данных

Связь между Unity и Python выполняется по локальному WebSocket-каналу ws://127.0.0.1:8765 [7, 14]. Такой вариант выбран из-за простоты настройки и отсутствия необходимости передавать данные через внешний сервер. Сообщения имеют формат JSON [14, 15].

Жизненный цикл теста включает несколько последовательных состояний:

StartScreen → *Connecting* → *Ready* → *Running* → *Analyzing* → *Result*.

После запуска Unity устанавливает WebSocket-соединение с Python-сервисом. Backend отправляет сообщение service_status, в котором указывается состояние сервера и доступность потока Emotiv. Если соединение успешно установлено, кнопка начала теста становится доступной.

После запуска смены Unity формирует сообщение session_start (см. рис. 3):

```
</> JSON

{
  "schema": "fatigue.v1",
  "type": "session_start",
  "session_id": "s-001",
  "duration_s": 60
}
```

Рисунок 3 – JSON сообщение о старте сессии

В течение теста приложение передаёт два типа событий. Поведенческое событие содержит идентификатор задания, время от начала сессии, время реакции и признак правильности ответа. JSON контракт представлен на рисунке 5.



Рисунок 4 – JSON сообщение о событиях в игре

Если оператор не успел обработать тревогу за отведённое время, регистрируется событие target_missed. Для виртуального взгляда используется отдельное сообщение gaze_event. Оно содержит координаты положения взгляда, объект под лучом камеры, длительность фиксации, разброс и признак отвлечения.

После завершения 60-секундной смены Unity отправляет сообщение session_finish. Python извлекает накопленные события, вычисляет сводные признаки и возвращает результат:



Рисунок 5 – JSON сообщение о результате анализа усталости

Для обработки сообщений в backend должна быть предусмотрена проверка структуры входных данных. Сервис контролирует наличие идентификатора сессии, временных значений и обязательных полей события. Некорректное сообщение не завершает работу сервера, а возвращает клиенту сообщение об ошибке. Завершённая сессия удаляется из списка активных, поэтому повторная обработка одного и того же идентификатора не должна влиять на уже рассчитанный результат.

2.2 Реализация тестовой виртуальной среды и регистрация поведения оператора

Тестовая среда реализована как настольная трёхмерная сцена от первого лица. Пользователь выполняет роль диспетчера городской энергосети. На рабочей панели отображаются текущая тревога, таймер, состояние смены и доступные действия. Дополнительная карта позволяет выбирать резервные линии (см. рис. 7).

JSON

```
{
  "schema": "fatigue.v1",
  "type": "session_result",
  "session_id": "s-001",
  "fatigue_score": 0.52,
  "class": "medium",
  "source": "live"
}
```

Рисунок 6 – JSON сообщение о результате анализа усталости

Продолжительность одной сессии составляет 60 секунд. Сценарий включает десять событий, распределённых по трём фазам с разной интенсивностью, для удобства они представлены в таблице 2.

Таблица 2 — Фазы теста

Фаза	**Интервал**	**Количество событий**	**Особенности**
Стабильная работа	0–16 с	2	редкие тревоги, увеличенный лимит реакции
Повышенная нагрузка	16–34 с	3	более частые события и сокращённое время ответа
Кризисный пик	34–60 с	5	короткий лимит, резервные линии и ложный сигнал

В начале каждой тревоги пользователю необходимо определить район и выбрать соответствующую линию (см. рис. 7). Для подтверждения действия используется повторное нажатие выбранной кнопки или клавиша Enter. Если выбрана неправильная линия, интерфейс показывает сообщение об ошибке, а локальный счётчик ошибок увеличивается. При правильном ответе тревога закрывается, а система переходит к следующему событию.

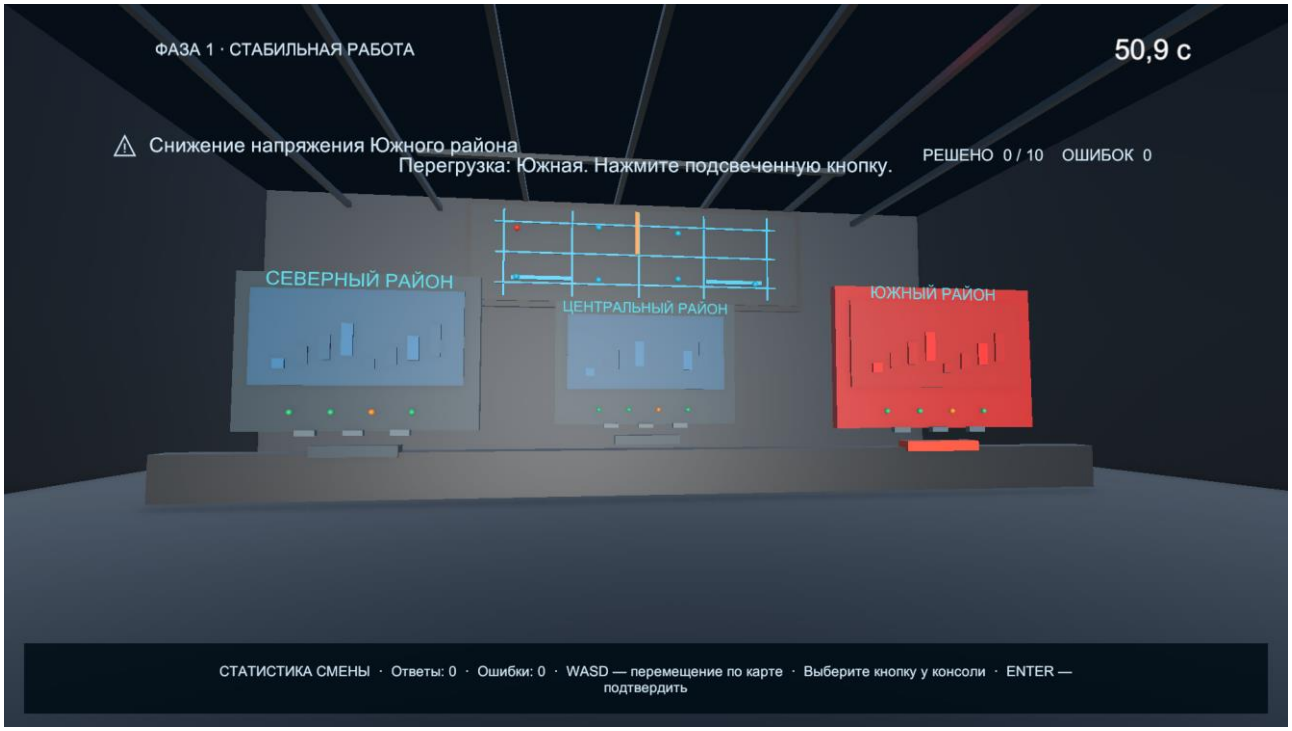


Рисунок 7 – Unity-сцена игры

Одно из заданий представляет собой ложный сигнал. В этом случае правильным действием является команда «Игнорировать». Такой вариант позволяет проверить не только скорость выбора, но и способность оператора отличать аварийное событие от обычного состояния оборудования.

Для каждого события задаётся собственный временной лимит. В первой фазе он составляет 8 секунд, во второй — 6 секунд, а в третьей — 4 секунды. При приближении к окончанию лимита цвет таймера меняется с обычного на жёлтый, а затем на красный. Дополнительно выводится текст «КРИТИЧЕСКОЕ ВРЕМЯ». Если время истекает, Unity отправляет событие target_missed и активирует следующую тревогу.

Время реакции рассчитывается от момента появления задания до его подтверждения. В аналитический сервис передаётся длительность реакции, идентификатор цели и признак правильного решения. Это позволяет учитывать не только сам факт ошибки, но и изменение скорости работы оператора на разных этапах смены.

Отдельный компонент GazeSimulator отвечает за регистрацию виртуального взгляда. Он использует камеру Unity и луч, направленный через текущую позицию мыши. Луч определяет объект, на который смотрит пользователь, а компонент сохраняет его имя, нормализованные координаты положения, расстояние перемещения курсора и длительность фиксации.

С заданным интервалом в одну секунду Unity формирует событие:

JSON

```
{
  "schema": "fatigue.v1",
  "type": "gaze_event",
  "session_id": "s-001",
  "t_s": 12.4,
  "target_id": "TargetNorth",
  "gaze_x": 0.51,
  "gaze_y": 0.44,
  "event": "fixation_end",
  "duration_s": 0.74,
  "dispersion": 0.08,
  "distracted": false
}
```

Рисунок 8 – JSON сообщение об взгляде пользователя

Показатель duration_s характеризует продолжительность фиксации на одном объекте. Значение dispersion отражает изменение положения курсора между соседними измерениями. Если луч направлен не на текущую тревогу, событие помечается как отвлечение.

Такой способ регистрации не является настоящей окулографией. Он не измеряет движения глаз и не заменяет eye-tracker. Его назначение состоит в проверке полного конвейера: генерации событий, их передачи по WebSocket, агрегации и включения в итоговый индекс.

В конце теста Unity отправляет сообщение о завершении сессии. После этого новые игровые события не формируются, а приложение ожидает ответ Python (см. рис. 9). Полученный результат сохраняется в памяти приложения и отображается на сцене Results.

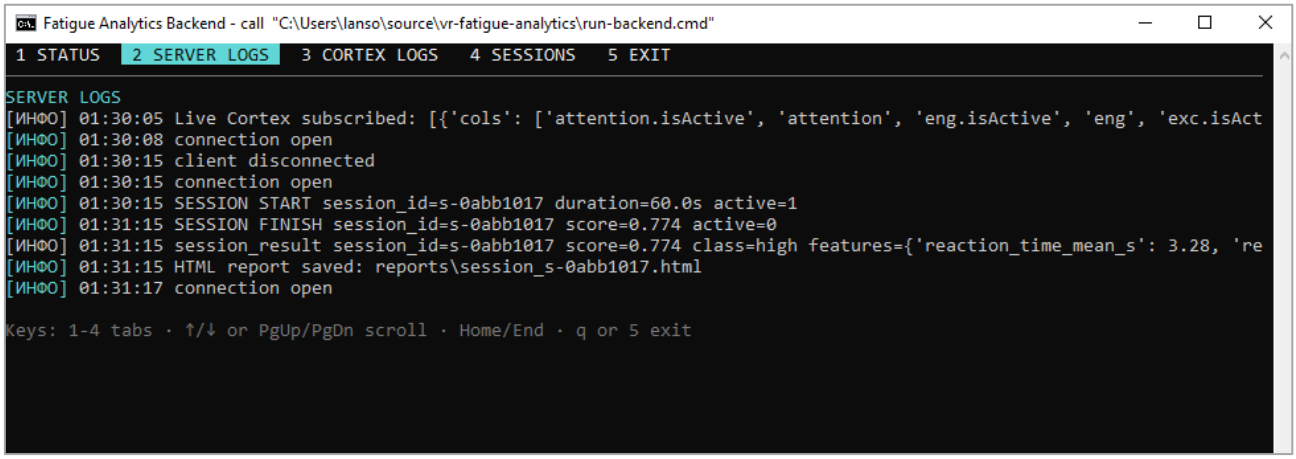


Рисунок 9 – Скриншот логов Python сервера

2.3 Обработка метрик Emotiv, расчет интегрального индекса и режимы работы системы

В текущей версии реализован один рабочий режим — Live. Он использует Emotiv Launcher, Cortex API и виртуальное устройство EPOC X. Перед запуском теста Python-сервис устанавливает защищённое соединение с локальным Cortex

по адресу wss://localhost:6868, выполняет авторизацию, находит подключённое устройство, создаёт сессию и подписывается на поток met.

Во время проверки 5 ноября 2026 года использовался виртуальный EPOC X с идентификатором EPOCX-3C0EF39. Версия Cortex составила 4.13.5.3. Для устройства были подтверждены частоты eegRate = 128 и memsRate = 32. Поток met возвращал следующие показатели: attention, eng, exc, lex, str, rel, int.

В сообщении Cortex каждому числовому показателю соответствовало поле активности isActive. Например, фактическое сообщение могло иметь следующий вид:

[true, 0.5, true, 0.5, true, 0.5, 0.0,
true, 0.5, true, 0.5, true, 0.5]

Адаптер cortex_metrics.py преобразует массив Cortex во внутренний словарь с именованными полями. В обработку попадают только подтверждённые показатели.

В течение сессии Python сохраняет полученные сообщения Emotiv вместе с событиями Unity. После получения session_finish выполняется агрегация. Для поведенческих данных рассчитываются:

- среднее время реакции;
- медианное время реакции;
- количество ошибочных ответов;
- количество пропущенных целей;
- количество поздних ответов.

Для событий взгляда вычисляются средняя длительность фиксации, средний разброс и количество отвлечений. Для Emotiv рассчитывается среднее значение каждого доступного поля и количество полученных сообщений.

Итоговый показатель строится как эвристическая комбинация трёх компонентов:

$$
\begin{aligned}
&fatigue_score = \\
&\quad 0.55 \times behavior_component + \\
&\quad 0.25 \times gaze_component + \\
&\quad 0.20 \times emotiv_component
\end{aligned}
$$

Если какой-либо источник данных отсутствует, используются только доступные компоненты, а их веса нормализуются. Это позволяет завершить анализ даже при временной недоступности части данных.

Поведенческий компонент определяется с учётом средней реакции, ошибок, пропусков и поздних ответов:

$$
\begin{aligned}
&behavior_component = \\
&\quad clamp(\\
&\qquad 0.55 \times reaction_factor + \\
&\qquad 0.30 \times error_factor + \\
&\qquad 0.15 \times missed_factor + \\
&\qquad 0.10 \times late_factor \\
&\quad)
\end{aligned}
$$

В качестве базового масштаба для времени реакции используется значение 3 секунды. Количество ошибок, пропусков и поздних ответов нормализуется относительно десяти событий теста.

Gaze-компонент учитывает три признака: разброс наведения, число отвлечений и длительность фиксации:

$$
\begin{aligned}
&gaze_component = \\
&\quad clamp(\\
&\qquad 0.60 \times dispersion + \\
&\qquad 0.25 \times distraction_factor + \\
&\qquad 0.15 \times fixation_factor
\end{aligned}
$$

)

Emotiv-компонент в текущей реализации использует средние значения eng и rel. При снижении вовлечённости и расслабления его значение увеличивается:

$$
\begin{aligned}
&emotiv_component = \\
&\quad clamp(\\
&\qquad 0.60 \times (1 - mean_eng) + \\
&\qquad 0.40 \times (1 - mean_rel) \\
&\quad)
\end{aligned}
$$

Полученный результат ограничивается диапазоном от 0 до 1 и относится к одной из трёх категорий:

- *low* — значение меньше 0,34;

- *medium* — значение от 0,34 до 0,67;

- *high* — значение от 0,67 и выше.

Эти границы и коэффициенты не получены в результате обучения на размеченной медицинской выборке. Они используются как прозрачная эвристика, позволяющая проверить работу программного конвейера и связать итог с исходными признаками.

Помимо общего индекса, Python отдельно рассчитывает статистику по фазам смены. Для интервалов 0–16, 16–34 и 34–60 секунд определяются количество ответов, ошибки, пропуски, точность и среднее время реакции. Такое разделение позволяет сравнить работу оператора в начале теста и в период повышенной нагрузки.

После завершения расчёта Python отправляет Unity сообщение session_result. Интерфейс показывает итоговый индекс, категорию состояния, длительность сессии, реакцию, ошибки, пропуски, поздние ответы, параметры взгляда, количество сообщений Emotiv и источник данных. Одновременно backend сохраняет подробный HTML-отчёт (см. рис. 10).

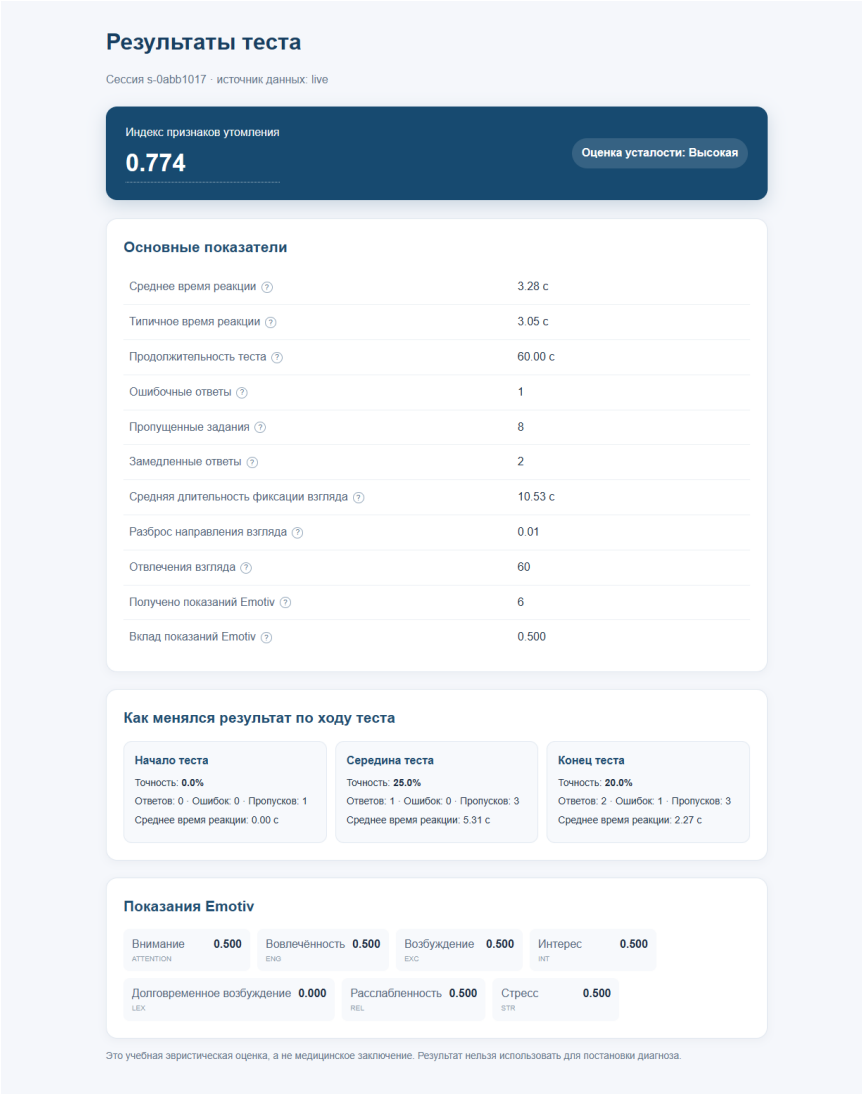


Рисунок 10 – Скриншот HTML отчета

Таким образом, реализованная система объединяет игровое поведение, моделируемые признаки взгляда и подтверждённые Performance Metrics Emotiv в едином аналитическом цикле. Полученный индекс используется только для оценки состояния в учебном прототипе. Он не является медицинским диагнозом и не подтверждает наличие клинической усталости.

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы был разработан прототип системы предиктивной аналитики состояния оператора в виртуальной среде, объединяющий поведенческие, окулографические и доступные нейрофизиологические признаки.

В теоретической части рассмотрены понятия утомления и когнитивной нагрузки, а также выделены три группы признаков с их ограничениями: поведение зависит от сложности задания, взгляд в текущей версии моделируется, а метрики Emotiv используются только после подтверждения фактического ответа Cortex. Показано, что надёжная оценка состояния возможна лишь при совместном учёте признаков разной природы.

В практической части реализована клиент-серверная архитектура: Unity отвечает за пользовательский интерфейс, сценарий событий и моделирование взгляда, а Python-сервис выполняет приём данных по WebSocket, агрегацию и расчёт. Разработана тестовая сцена 60-секундной смены с тремя фазами нарастающей нагрузки, регистрацией времени реакции, ошибок, пропусков и поздних ответов. Реализован адаптер Cortex, преобразующий подтверждённые метрики потока met во внутренний формат, и эвристический расчёт интегрального индекса с нормализацией весов при отсутствии части источников.

Полученный результат позволяет воспроизвести расчёт, связать итоговую оценку с входными признаками и проследить изменение качества работы оператора по фазам смены. При этом индекс является оценкой учебного прототипа и не рассматривается как медицинское заключение.

Дальнейшее развитие работы связано с подключением реального eye-tracker, использованием сырых EEG-каналов, онлайн-адаптацией сложности и заменой эвристического расчёта моделью, обученной на накопленных данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Salomone M., Audiffren M. et al. Overview and Perspectives on the Assessment and Mitigation of Cognitive Fatigue in Operational Settings // Cyber–Physical–Human Systems. – Wiley, 2023.
2. Longo L., Wickens C.D., Hancock G., Hancock P.A. Human Mental Workload: A Survey and a Novel Inclusive Definition // Frontiers in Psychology. – 2022. – Vol. 13.
3. Matulewski J., Meszyński S., Sokolov O. et al. Searching for Eye Movement Correlates of Mental Fatigue When Working with Gaze-Controlled Software Using Low-Cost Eye Trackers // Proceedings of the ACM Conference. – 2026.
4. Bafna T., Hansen J.P. Mental fatigue measurement using eye metrics: A systematic literature review // Psychophysiology. – 2021. – Vol. 58, No. 6. – Art. e13828.
5. Emotiv. Performance Metrics. – URL: https://account.emotiv.com/performance-metrics/ (дата обращения: 06.11.2026).
6. Zhang D., Shi J. Research on Assessment Method for Cognitive Influence Factors of Nuclear Power Plant Operators Based on Fuzzy Analytic Hierarchy Process and Deep Learning of Electroencephalogram Signals // Nuclear Science and Engineering. – 2025. – Vol. 199, No. 6. – P. 838–853.
7. StressLens: Real-Time Stress Detection and Monitoring Platform in Virtual Reality // Zenodo. – 2026.
8. Lightweight EEG-Based Fatigue Detection Framework for Remote Tower Air Traffic Controllers // IEEE Xplore. – 2025.
9. Emotiv. Cortex API Documentation. – URL: https://emotiv.gitbook.io/cortex-api (дата обращения: 06.11.2026).
10. Emotiv. Emotiv Launcher Documentation. – URL: https://emotiv.gitbook.io/emotiv-launcher (дата обращения: 06.11.2026).
11. Unity Technologies. Unity User Manual 6.0. – URL: https://docs.unity3d.com/Manual/ (дата обращения: 06.11.2026).

12. Unity Technologies. Universal Render Pipeline Documentation. – URL: https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@17.0/manual/ (дата обращения: 06.11.2026).
13. Unity Technologies. Input System Documentation. – URL: https://docs.unity3d.com/Packages/com.unity.inputsystem@1.20/manual/ (дата обращения: 06.11.2026).
14. RFC 6455. The WebSocket Protocol / I. Fette, A. Melnikov. – IETF, 2011. – URL: https://www.rfc-editor.org/rfc/rfc6455 (дата обращения: 06.11.2026).
15. Newtonsoft. Json.NET Documentation. – URL: https://www.newtonsoft.com/json (дата обращения: 06.11.2026).

Курсовую работу подготовил студент

Казаккулов У.Ф.

(подпись, фамилия, инициалы)